Heston Model Calibration with Gaussian Process & Carr-Madan Fast Fourier Transform

Dhital, Ayush; Gabe Michael; Zheng, Chloe

July 2026

Contents

1 Introduction **2**

2 Heston Model **2**
2.1 Model Specification 2
2.2 Stock Paths and volatility in Heston Model 4
2.2.1 Mean-Reversion Rate (κ) 4
2.2.2 Long-Run Variance (θ) 4
2.2.3 Stock-Volatility correlation (ρ) 6

3 Carr-Madan FFT Setup for Heston Model **8**
3.1 Setup: Modified Call Price and Its Fourier Transform 9
3.2 The Heston Characteristic Function 10
3.3 A quick Heston-FFT vs BSM/Monte-Carlo comparison 10

4 Gaussian Process (GP) **10**
4.1 Gaussian Process Emulator 10
4.2 Sparse Gaussian Process 11

5 Synthetic Data **12**
5.1 Sampling 12
5.2 Generating the Synthetic Implied Volatility Surface 14

6 Calibration **15**
6.1 Data 16
6.2 Differences Between FFT, GP, and Sparse GP Calibration 17

7 Results **19**

1 Introduction

The Black–Scholes-Merton (BSM) model is a foundational option-pricing framework, but its assumption of constant volatility limits its ability to reproduce features observed in real markets, such as the volatility smile. This project therefore considers the Heston model, in which the asset price and its variance follow coupled stochastic processes, allowing it to capture the skew that BSM structurally cannot. We use the Carr-Madan FFT method as our primary pricing benchmark: it prices the Heston model semi-analytically rather than through simulation, giving an accurate and fast reference.

Calibrating the Heston model against real market data is computationally expensive: each iteration of a gradient-based optimizer requires repeatedly re-evaluating the pricer across the full option chain. Therefore, we develop Gaussian Process (GP) and sparse GP emulators that learn the mapping from Heston parameters and option characteristics to implied volatility offline, so that calibration can query the emulator rather than the pricer directly. This approach follows Mongwe et al. (2025) [4], who demonstrate that GP-based calibration can match the accuracy of standard techniques while additionally producing calibrated uncertainty estimates at each point on the volatility surface - a property neither FFT nor Monte Carlo pricing provides natively. Motivated by this, we evaluate GP and sparse GP emulators along two axes suggested by the textbook's framework: computational speed relative to direct FFT and Monte Carlo calibration, and the extent to which their predictive uncertainty correctly identifies regions of the SPY options chain where calibration is less reliable, such as short-dated, thinly-quoted contracts.

As shown in Figure 1, the project workflow consists of generating synthetic Heston data, training and validating the emulators using an 80/20 data split, preparing a real SPY options dataset, calibrating the model using GP, sparse GP, FFT, and Monte Carlo methods, and comparing their accuracy, computational efficiency, and predictive uncertainty.

2 Heston Model

2.1 Model Specification

The Heston model [3] is a stochastic volatility model in which the asset price S_t and its instantaneous variance v_t evolve jointly according to the coupled system of stochastic differential equations

$$dS_t = \mu S_t \, dt + \sqrt{v_t} \, S_t \, dW_t^S, \tag{1}$$

$$dv_t = \kappa(\theta - v_t) \, dt + \sigma \sqrt{v_t} \, dW_t^v, \tag{2}$$

with the two driving Brownian motions correlated according to

$$dW_t^S \, dW_t^v = \rho \, dt. \tag{3}$$

Unlike the Black-Scholes-Merton model, in which volatility is a fixed constant, the Heston model allows volatility to evolve as its own random process, coupled to the asset price. A

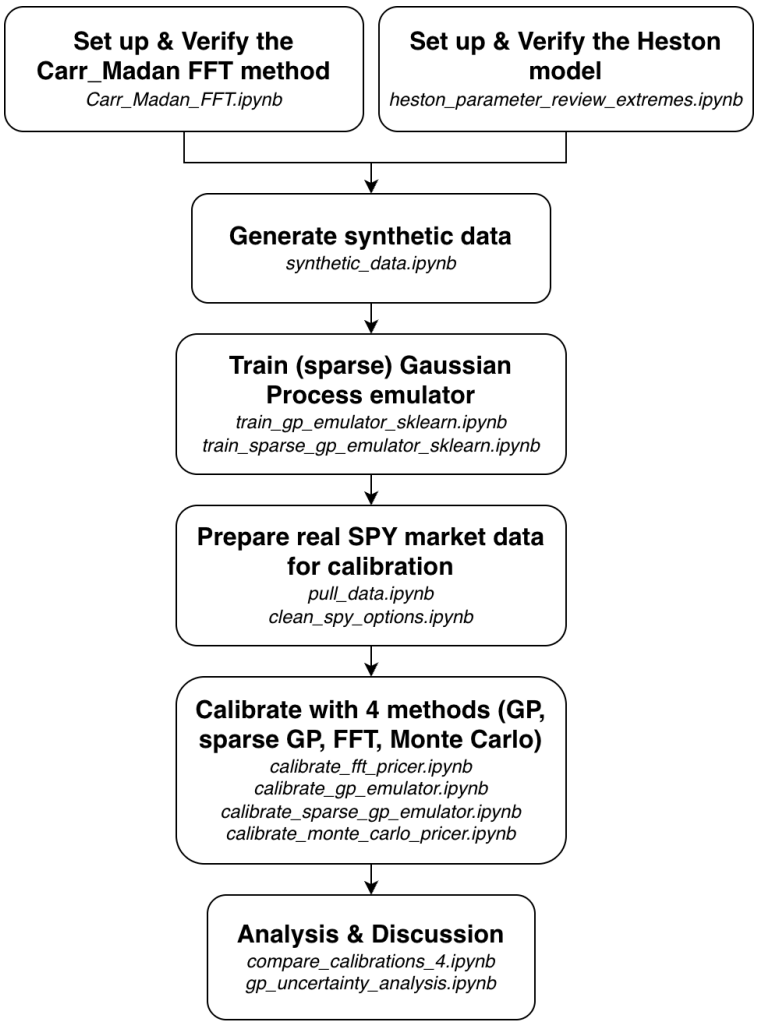

Figure 1: Workflow of the project

subtlety of this model is that the variance v_t remains strictly positive for all t, only when the parameters satisfy the *Feller condition* [2]

$$2\kappa\theta \geq \sigma^2. \tag{4}$$

When this condition is violated, v_t can reach zero, though the mean-reverting drift term prevents it from being absorbed there. In all of our subsequent analysis, we make sure that this condition is never violated by manually fixing the initial condition of the market under Heston model. The Heston model has 5 unknown parameters that describe how a stock's volatility behaves over time:

- $\boldsymbol{v_0}$ - the initial variance, setting the starting level of volatility;
- $\boldsymbol{\theta}$ - the long-run variance, the level toward which v_t reverts;
- $\boldsymbol{\kappa}$ - the mean-reversion speed;

- $\boldsymbol{\sigma}$ - the volatility of variance;
- $\boldsymbol{\rho}$ - the correlation between asset-price and variance innovations.

2.2 Stock Paths and volatility in Heston Model

From equation (1), (2), and (3), we see that Heston Model has five sets of parameters. Given an initial condition (starting 5 sets of parameters), one can model a stock path with this model. In this section, we see how each parameter changes the stock path and what they mean. In all subsequent subsection, our initial parameter values are (unless stated otherwise):

1. $S_0 = 100$: Initial stock price
2. $\nu_0 = 0.08$: Initial stock volatility
3. $\theta = 0.04$: Mean volatility
4. $\kappa = 2.5$: Mean-reversion rate
5. $\sigma = 0.06$: Variance of volatility (vol-of-vol)
6. $\rho = -0.7$: Stock-volatility correlation
7. $\mu = 0$: The drift term
8. Step size = 504 (2 years)

2.2.1 Mean-Reversion Rate (κ)

The parameter κ controls how quickly the instantaneous variance v_t reverts toward its long-run level θ, on average. This is the Heston model's analogue of the constant-volatility assumption in the BSM model: as $\kappa \to \infty$ (or equivalently, as the vol-of-vol parameter $\sigma \to 0$), v_t is pinned arbitrarily close to θ for all t, and the model collapses toward BSM-like dynamics with a single, effectively constant volatility level.

Figures 2 and 3 compare simulated paths for $\kappa = 0.25$ and $\kappa = 8$, holding all other parameters fixed. With a small mean-reversion rate, the latent variance can remain far from θ for extended periods, since the pull back toward the long-run level is weak. With a larger mean-reversion rate, v_t is pulled back toward θ much more quickly and crosses it far more frequently, staying tightly clustered around the long-run level throughout the simulation.

2.2.2 Long-Run Variance (θ)

The parameter θ sets the long-run volatility, towards which the instantaneous variance v_t reverts, and can be interpreted as the average variance the asset is expected to exhibit over long horizons. Since v_t continually reverts toward θ (at rate κ), increasing θ raises the level around which the latent variance fluctuates, without necessarily changing how far v_t wanders from that level at any given time. The spread is governed instead by κ and the vol-of-vol

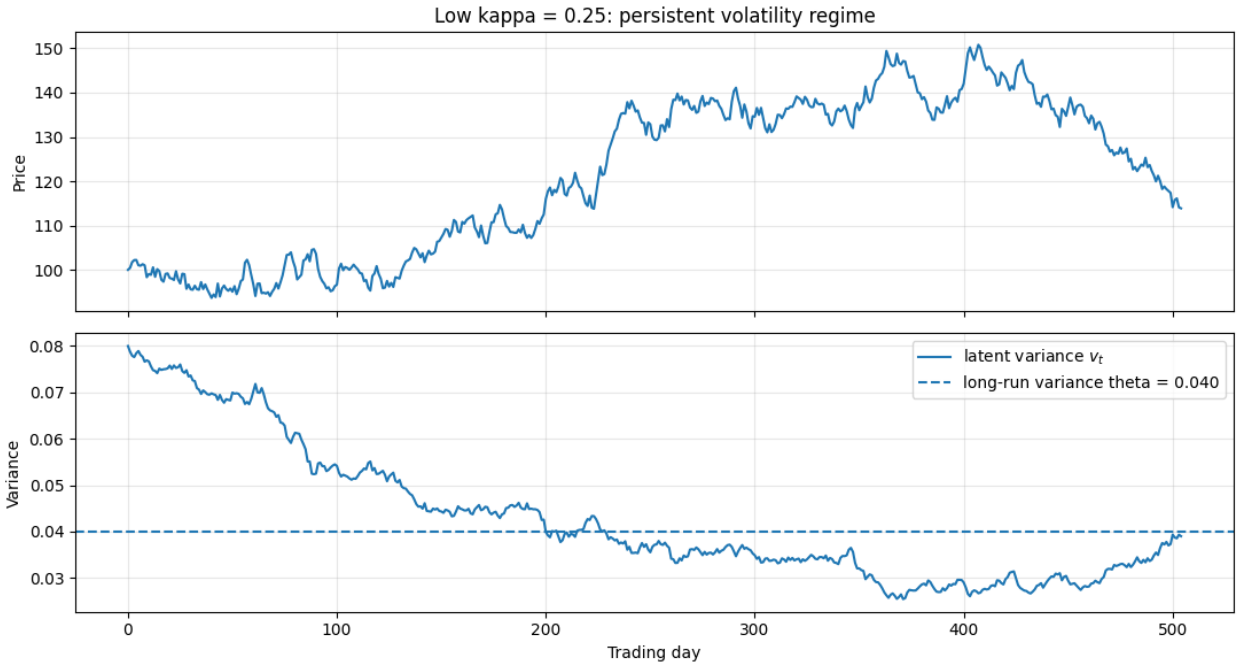

Figure 2: Stock price and latent variance paths for $\kappa = 0.25$. With slow mean reversion, v_t drifts away from θ for long stretches of time.

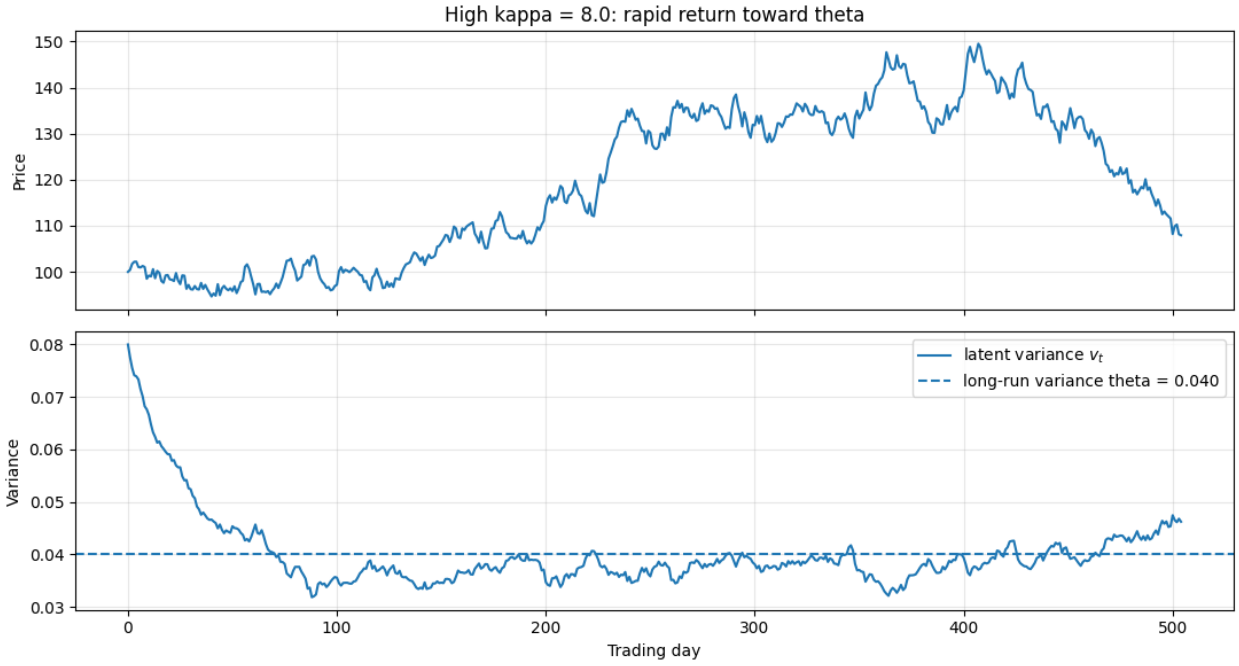

Figure 3: Stock price and latent variance paths for $\kappa = 8$. With fast mean reversion, v_t crosses θ far more often and remains close to it throughout.

parameter σ. A higher θ correspondingly widens the typical range of stock-price fluctuations, since the asset spends more time exposed to higher variance and the resulting price path

exhibits larger swings on average.

Figures 4 and 5 compare simulated paths for $\theta = 0.01$ (10% long-run volatility) and $\theta = 0.16$ (40% long-run volatility), holding all other parameters fixed. Since v_0 differs from θ in both cases, the latent variance can be seen adjusting away from its starting value and toward the respective long-run level early in each path, before settling into fluctuations around θ. Also, it is clear from Figures 4 and 5 that a higher-θ path exhibits larger stock-price swings.

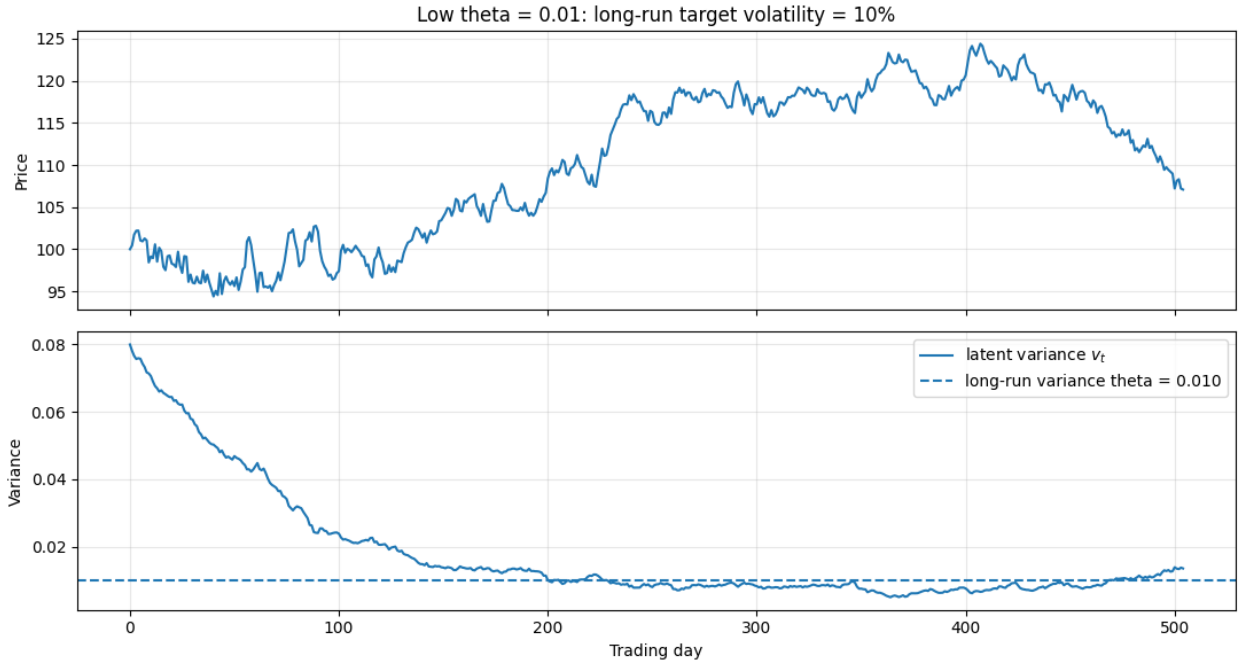

Figure 4: Stock price and latent variance paths for $\theta = 0.01$ (10% long-run volatility), with $v_0 = 0.08$. Variance drifts downward from its initial level toward the lower long-run target.

2.2.3 Stock-Volatility correlation (ρ)

The parameter ρ governs the instantaneous correlation between the stock price and its variance. When ρ is strongly negative, downward moves in the stock price tend to coincide with upward jumps in variance. When $\rho = 0$, stock price and variance are independent. When ρ is strongly positive, the effect reverses: rising prices tend to coincide with rising variance.

Figures 6–8 compare simulated paths for $\rho = 0.95$, $\rho = 0$, and $\rho = -0.95$, holding all other parameters fixed. The variance paths themselves look similar across the three cases, since ρ does not affect the dynamics of v_t in isolation; the difference instead shows up in how the stock price co-moves with the variance.

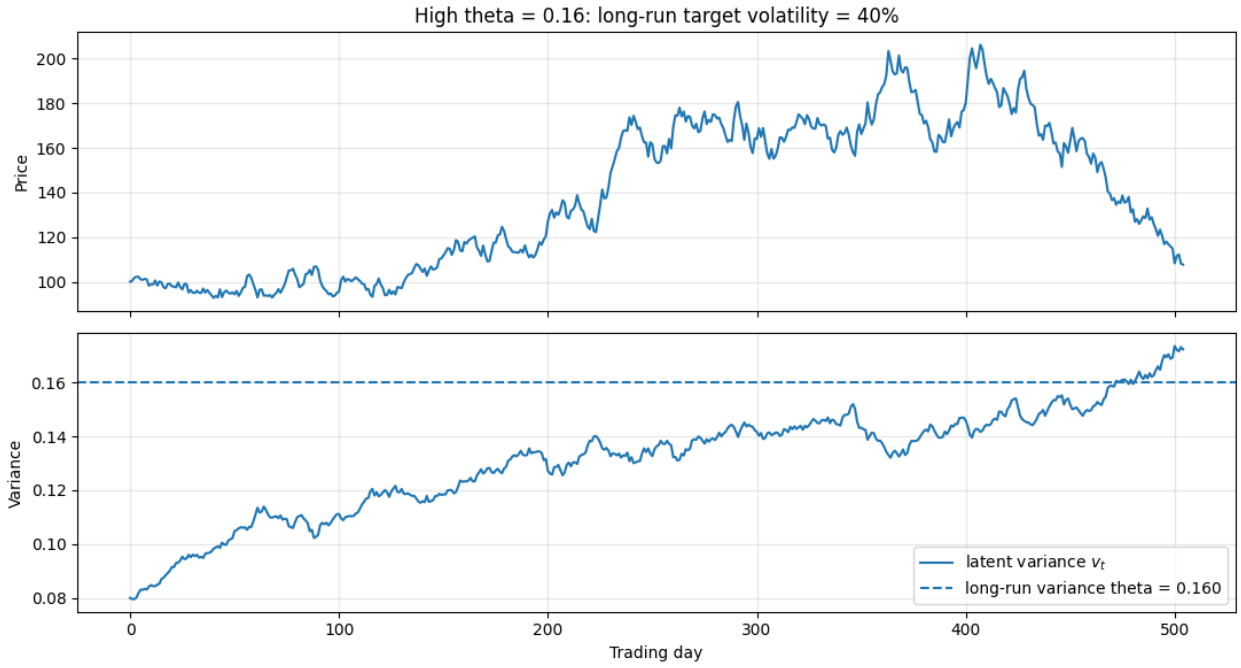

Figure 5: Stock price and latent variance paths for $\theta = 0.16$ (40% long-run volatility), with $v_0 = 0.08$. Variance drifts upward from its initial level toward the higher long-run target, producing larger stock-price swings.

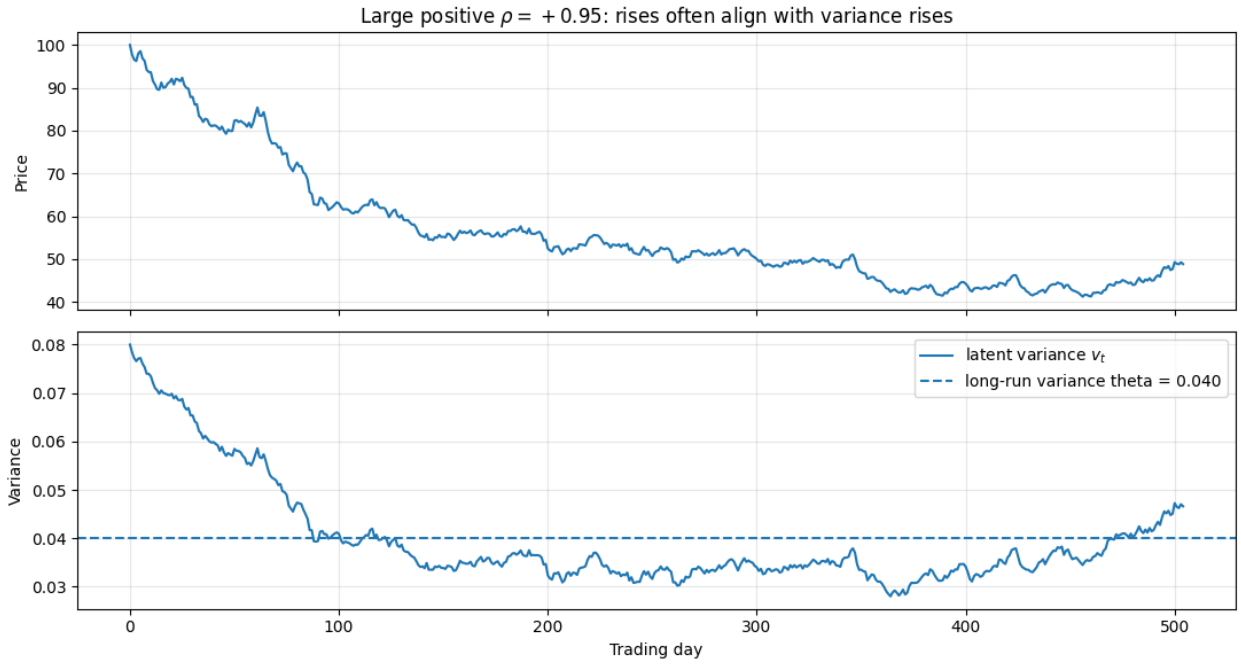

Figure 6: Stock price and latent variance paths for $\rho = 0.95$. Rising variance tends to coincide with rising stock prices.

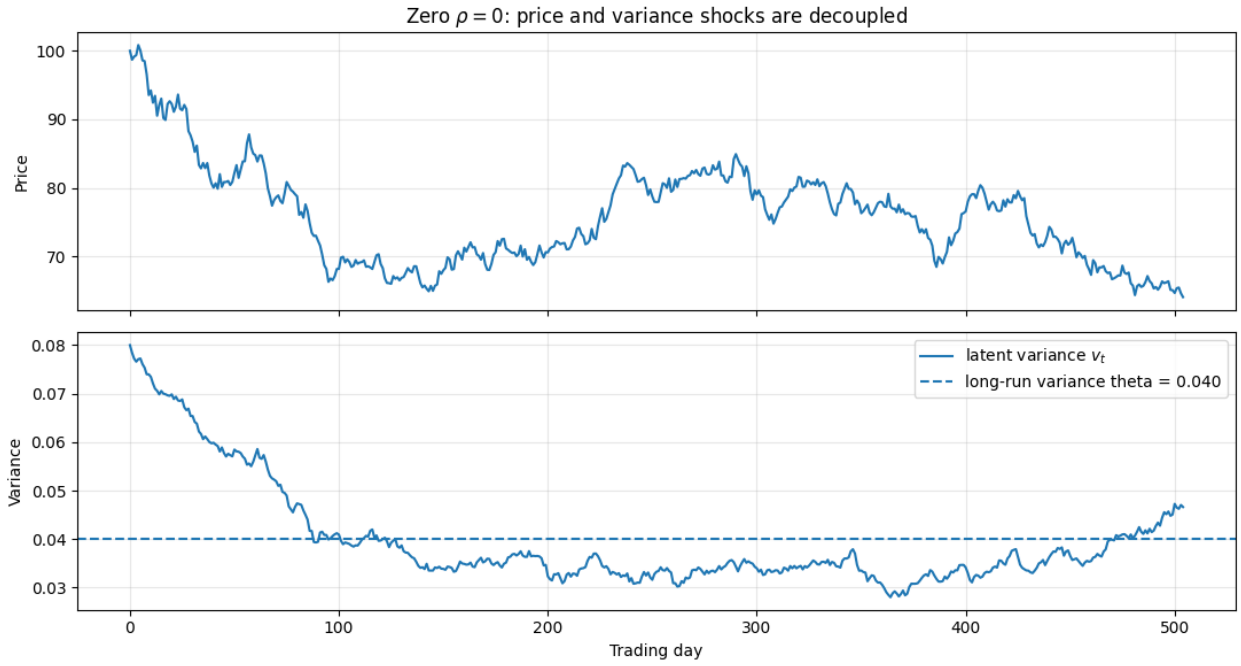

Figure 7: Stock price and latent variance paths for $\rho = 0$. Stock price and variance are independent, with no correlation between them.

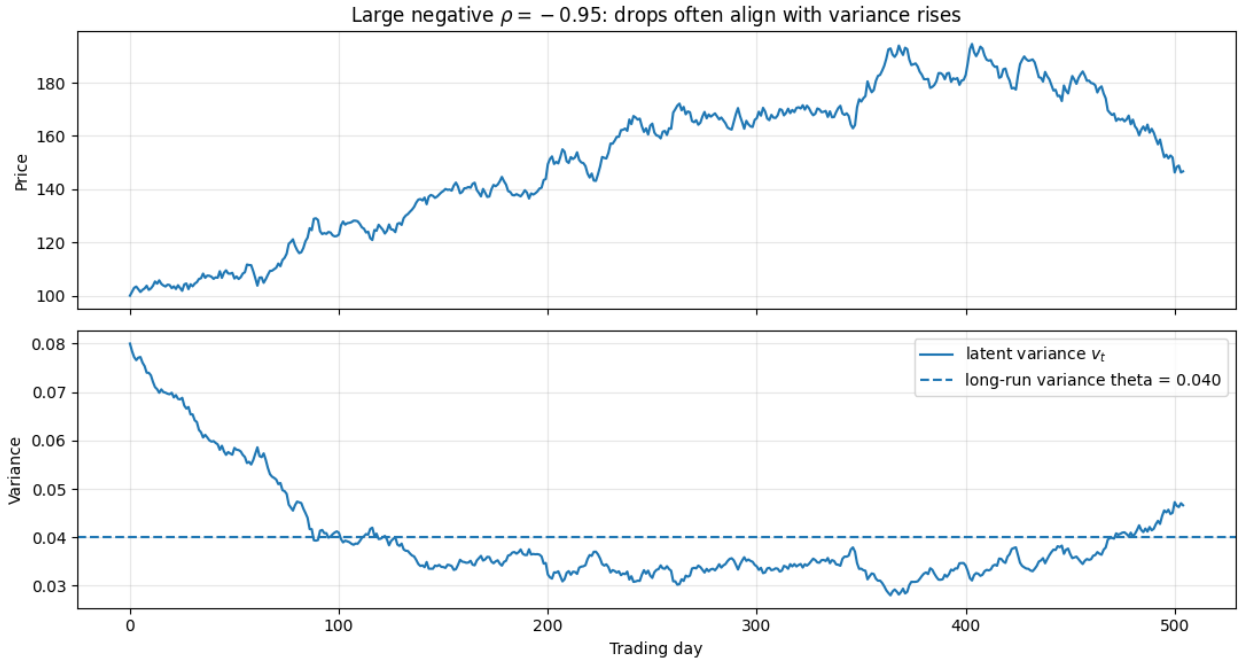

Figure 8: Stock price and latent variance paths for $\rho = -0.95$. Spikes in variance tend to coincide with drops in the stock price.

3 Carr-Madan FFT Setup for Heston Model

The BSM model is widely used to price options given a strike price and time to maturity, under the assumption that volatility is constant. The Heston model relaxes this assumption

by letting volatility itself follow a stochastic process, which better captures observed market features such as volatility smiles and skews. Here, we briefly review the Fast Fourier Transform (FFT) method of Carr and Madan [1] for pricing European options given a model's characteristic function, and specialize it to the Heston model.

3.1 Setup: Modified Call Price and Its Fourier Transform

Let $k = \ln K$ denote the log-strike price, and let $C_T(k)$ denote the price of a T-maturity European call struck at e^k. Writing $s_T = \ln S_T$ for the log stock price at maturity and $q_T(s)$ for its risk-neutral density, the characteristic function of s_T is defined as

$$\phi_T(u) \equiv E^Q\left[e^{ius_T}\right] = \int_{-\infty}^{\infty} e^{ius}\, q_T(s)\, ds, \tag{5}$$

and the call price can be written directly as a discounted expectation of the payoff,

$$C_T(k) = e^{-rT} \int_k^{\infty} (e^s - e^k)\, q_T(s)\, ds. \tag{6}$$

As $k \to -\infty$, $C_T(k) \to S_0$, a nonzero constant; consequently $C_T(k)$ is not square-integrable in k over the whole real line, and its Fourier transform does not exist in the usual sense. Carr and Madan's resolution [1] is to instead work with a *damped* call price,

$$c_T(k) \equiv e^{\alpha k}\, C_T(k), \qquad \alpha > 0, \tag{7}$$

whose exponential decay as $k \to -\infty$ restores square-integrability, and to Fourier transform this modified quantity instead:

$$\psi_T(v) = \int_{-\infty}^{\infty} e^{ivk}\, c_T(k)\, dk. \tag{8}$$

Substituting the definitions (6) and (7) into (8) and swapping the order of integration gives

$$\psi_T(v) = \int_{-\infty}^{\infty} e^{ivk} e^{\alpha k} e^{-rT} \int_k^{\infty} (e^s - e^k)\, q_T(s)\, ds\, dk \tag{9}$$

$$= e^{-rT} \int_{-\infty}^{\infty} q_T(s) \int_{-\infty}^{s} \left(e^{s+\alpha k} - e^{(1+\alpha)k}\right) e^{ivk}\, dk\, ds. \tag{10}$$

The inner integral over k is elementary, yielding terms of the form $e^{(\alpha+1+iv)s}/(\alpha+iv)$ and $e^{(\alpha+1+iv)s}/(\alpha+1+iv)$; combining them and recognizing the remaining integral over s as ϕ_T evaluated at a shifted argument yields the closed form

$$\psi_T(v) = \frac{e^{-rT}\, \phi_T\big(v - (\alpha+1)i\big)}{\alpha^2 + \alpha - v^2 + i(2\alpha+1)v}. \tag{11}$$

This is the central result of the Carr-Madan method: it expresses the transform of the option price entirely in terms of the characteristic function ϕ_T of the log-price. Since $C_T(k)$ is real, $\psi_T(v)$ is Hermitian (even real part, odd imaginary part), so the inverse Fourier transform can be folded onto the positive half-line, recovering the call price as

$$C_T(k) = \frac{e^{-\alpha k}}{\pi} \int_0^{\infty} e^{-ivk}\, \psi_T(v)\, dv. \tag{12}$$

3.2 The Heston Characteristic Function

Equation (12) holds for any model with a known characteristic function; specializing to Heston, $\phi_T(u)$ is given in closed form by [3]

$$\phi_T(u) = \exp\big(iu \ln S_0 + C(u,T) + D(u,T)\, v_0\big), \tag{13}$$

where

$$D(u,T) = \frac{\kappa - \rho\sigma iu - d}{\sigma^2} \cdot \frac{1 - e^{-dT}}{1 - g\, e^{-dT}}, \tag{14}$$

$$C(u,T) = iuT\, r + \frac{\kappa\theta}{\sigma^2}\left[(\kappa - \rho\sigma iu - d)T - 2\ln\left(\frac{1 - g\, e^{-dT}}{1-g}\right)\right], \tag{15}$$

with

$$d = \sqrt{(\rho\sigma iu - \kappa)^2 + \sigma^2(iu + u^2)}, \qquad g = \frac{\kappa - \rho\sigma iu - d}{\kappa - \rho\sigma iu + d}. \tag{16}$$

Here $\kappa, \theta, \sigma, \rho, v_0$ are the Heston parameters introduced in Section 2, and r is the risk-free rate. We take the underlying stock to be non-dividend-paying, consistent with the stock dynamics in (1); a dividend yield q can be incorporated by replacing r with $r - q$ throughout if needed. Substituting (13) into (12) recovers the Heston call price as a function of strike and maturity.

3.3 A quick Heston-FFT vs BSM/Monte-Carlo comparison

Heston model is a generalized version of BSM model. If the parameter σ (vol-of-vol) is set to zero, and θ is set to initial volatility, then the Heston model reduces to BSM model. Here, we compare the price of a European call option between a BSM model and Heston model using Carr-Madan FFT method setting $\sigma = 0$ and $\theta = \nu_t$ for different strike prices (all maturity time is set to 1 year). This is implemented in our code `Carr_Madan_FFT.ipynb`. We have also simulated 120,000 realizations of stock path using Heston model and computed the expected price of call option at maturity (which we refer to as Monte Carlo result). We compare the Monte Carlo result with the Carr-Madan FFT method and compute the error. We verify that our FFT computation is reliable and the absolute difference in call option value at maturity between MC and FFT is 0.0286. This is shown in Figure 9 below.

4 Gaussian Process (GP)

4.1 Gaussian Process Emulator

Pricing an option under the Heston model with the Carr-Madan FFT method is fast, but not instant. If we want to explore thousands of different combinations of Heston parameters, strikes, and maturities, those individual calculations add up. A Gaussian Process (GP) offers a shortcut: instead of recomputing the implied volatility from scratch every time, we can train a GP model to *learn* the relationship between the inputs (the Heston parameters, strike, and maturity) and the output (implied volatility), based on a large set of examples

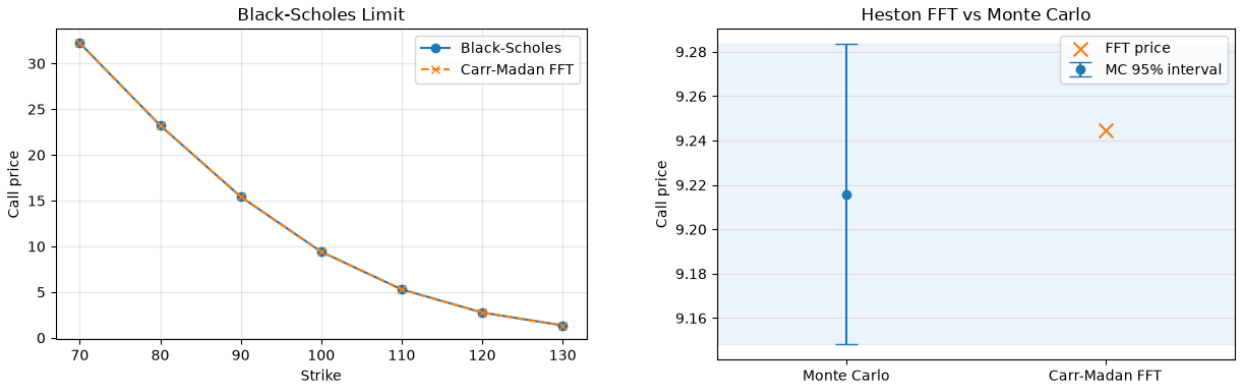

Figure 9: Comparing the price of a European Call option between Black-Scholes and Heston by setting the Heston parameters to yield the BS model. The price is computed using the Carr-Madan FFT in Heston model. We also compare this with Monte Carlo simulation (right) and the results from different methods all agree within uncertainty, thereby validating our FFT model.

we've already computed. Once trained, this model, often called an *emulator*, can predict the implied volatility for a new combination of inputs almost instantly, without rerunning the full pricing calculation.

What makes a GP different from many other prediction methods is that it doesn't just give a single best guess; it also tells us how confident it is in that guess. Intuitively, a GP works by comparing a new input to all the training examples it has seen: if the new input is similar to points it was trained on, it predicts with high confidence; if the new input lies in an unfamiliar region of parameter space (say), an unusual combination of Heston parameters, the model's prediction becomes less certain. This built-in uncertainty estimate is one of the most useful features of a GP, since it helps flag exactly where the emulator's predictions should be trusted less.

The tradeoff is computational cost. In order to make predictions, a GP essentially needs to compare every new point against every training point it was given, and this comparison becomes expensive very quickly as the number of training examples grows.

4.2 Sparse Gaussian Process

A sparse Gaussian Process is a variation designed to get around the computational cost problem so that we don't have to throw away most of our data. Instead of comparing a new point against every single training example, a sparse GP first summarizes the training data using a much smaller set of representative "landmark" points, called *inducing points*. Rather than picking these landmarks by hand, the model learns where to place them during training, positioning them wherever they best summarize the overall shape of the data. Predictions are then made by comparing new inputs only against this small set of landmarks, rather than the full training set, which makes both training and prediction dramatically faster.

This approximation does involve a tradeoff: because the model is working from a com-

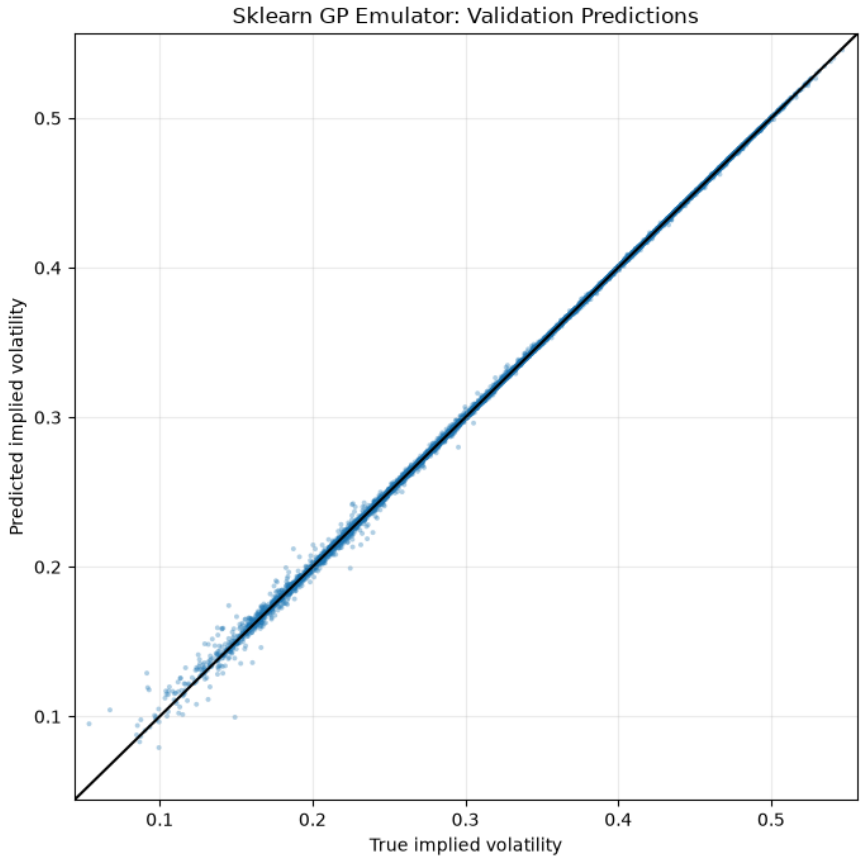

Figure 10: Predicted vs True IV from a GP emulator. The emulator was trained on 41,493 simulated data and we used 10,374 data sets for validation.

pressed summary of the data rather than the full set of examples, its predictions are typically slightly less accurate than those of an exact GP trained on the same points. In practice, though, this loss in accuracy tends to be modest, and the benefit is substantial: a sparse GP can be trained on the entire dataset instead of a small subsample, so it gets to learn from far more data overall. We later compare the sparse GP's predictions against the exact GP's to see how this tradeoff between using more data and losing some precision plays out in practice. Also, we see a clear distinction in overall computation time.

5 Synthetic Data

5.1 Sampling

The core problem with plain uniform random sampling in higher dimensions: even though each point is "random," pure chance means some regions of the space get clustered with many nearby points while other regions get almost none - purely by luck. This gets worse as dimensions increase (this is part of the "curse of dimensionality").
What Latin Hypercube Sampling (LHS) does differently: it divides each parameter's range

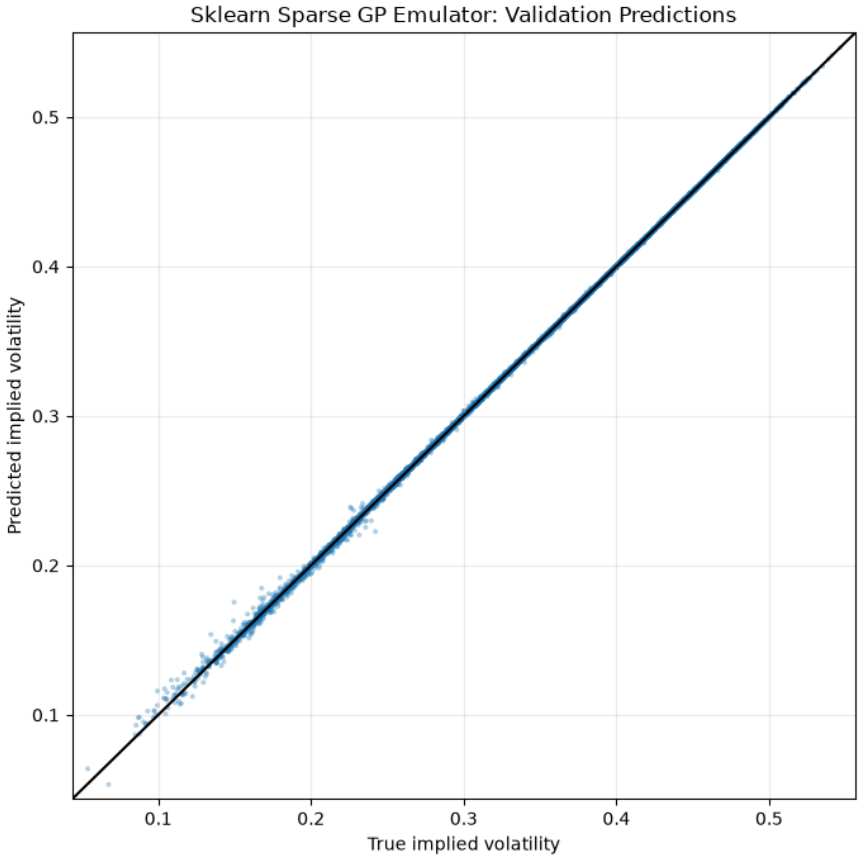

Figure 11: Predicted vs True IV from a sparse GP emulator. The emulator was trained on 41,493 simulated data and we used 10,374 data sets for validation.

into N equal-probability bins (N = number of samples), and guarantees exactly one sample per bin per dimension. This forces even coverage along every single axis, so you can't accidentally end up with, say, no samples at all in the low-κ range. It's not fully random, but it's far more evenly spread than uniform random, using the same number of samples.
The fundamental difference summary: uniform random sampling doesn't optimize for coverage, it just draws independently and hopes it averages out; space-filling designs like LHS deliberately construct the sample set to cover the space evenly.
What we actually want for this project: a space-filling design - since the GP emulator needs to learn the pricing function reasonably well everywhere in parameter space (you don't know in advance which region the real-data calibration will land in), evenly-spread samples are much more valuable per-sample than randomly clustered ones.
For using LHS: `scipy.stats.qmc.LatinHypercube` - one-line calls in scipy, so there's no real cost to using one over uniform random.

5.2 Generating the Synthetic Implied Volatility Surface

After sampling Heston parameter sets, we use each sampled parameter vector to generate a synthetic implied-volatility surface. Each sampled parameter set is of the form

$$\boldsymbol{\theta}_j = (v_{0,j}, \kappa_j, \theta_j, \sigma_{v,j}, \rho_j)\,,$$

where j indexes the sampled Heston parameter set. In our notebook, we generate 1,500 such parameter sets using Latin Hypercube Sampling.

For each sampled parameter set, we price European call options over a fixed grid of strikes and maturities. The notebook uses a normalized underlying price

$$S_0 = 100,$$

with risk-free rate

$$r = 0.03$$

and dividend yield

$$q = 0.$$

Instead of choosing strikes directly, we choose a moneyness grid (strike values relative to the current stock price)

$$\frac{K}{S_0} \in [0.70, 1.30],$$

so that the corresponding strikes are

$$K = S_0 \times \text{moneyness}.$$

The notebook uses 13 moneyness values between 0.70 and 1.30. The maturity grid is chosen in calendar days and then converted into years:

$$T \in \{2, 4, 7, 10, 13, 21, 30, 60, 90, 180, 365\}\,/365.$$

Thus, for each Heston parameter set, the notebook prices options across a grid of strikes and maturities. Since there are 13 moneyness values and 11 maturities, each parameter set can generate up to

$$13 \times 11 = 143$$

synthetic option observations.
For a given parameter set $\boldsymbol{\theta}_j$, strike K_i, and maturity T_i, the full synthetic input is

$$\mathbf{x}_{ij} = \begin{pmatrix} v_{0,j} \\ \kappa_j \\ \theta_j \\ \sigma_{v,j} \\ \rho_j \\ K_i \\ T_i \end{pmatrix}.$$

The FFT pricer is then used to compute the corresponding Heston call price:

$$C_{\text{Heston}}(\mathbf{x}_{ij}).$$

This call price is converted into a Black–Scholes implied volatility by solving

$$C_{\text{BS}}\left(S_0, K_i, T_i, r, q, \sigma\right) = C_{\text{Heston}}(\mathbf{x}_{ij})$$

for σ. The resulting solution is saved as the synthetic implied volatility:

$$IV_{\text{synthetic}}(\mathbf{x}_{ij}).$$

Therefore, each row of the synthetic dataset contains the sampled Heston parameters, the option contract inputs, the model-generated call price, and the corresponding implied volatility:

$$\left(v_0, \kappa, \theta, \sigma_v, \rho, K, T, C_{\text{Heston}}, IV_{\text{synthetic}}\right).$$

Rows with invalid prices or failed implied-volatility inversions are removed before saving the dataset.
The purpose of this synthetic dataset is to train the emulator to learn the map

$$(v_0, \kappa, \theta, \sigma_v, \rho, K, T) \mapsto IV_{\text{synthetic}}.$$

Once this map is learned, the emulator can be used during calibration as a fast approximation to the Heston implied-volatility surface.

6 Calibration

After training with synthetic data, we calibrate with real data.

1. First, we guess the five Heston parameters:

$$v_0,\ \kappa,\ \theta,\ \sigma_v,\ \rho.$$

For each real option contract i, we also take the known market inputs K_i and T_i. Therefore, the full input to the emulator is

$$\mathbf{x}_i = \begin{pmatrix} v_0 \\ \kappa \\ \theta \\ \sigma_v \\ \rho \\ K_i \\ T_i \end{pmatrix}.$$

The emulator then outputs the model-implied volatility,

$$IV_{\text{model}}(\mathbf{x}_i).$$

2. We compare the model-implied volatility to the real market-implied volatility:

$$IV_{\text{model}}(\mathbf{x}_i) - IV_{\text{market}}(K_i, T_i).$$

In the code, we use SciPy's `minimize` function to minimize the total squared error over all real option contracts:

$$\sum_{i=1}^{N} \left[IV_{\text{model}}(\mathbf{x}_i) - IV_{\text{market}}(K_i, T_i)\right]^2.$$

3. The optimizer repeats this process by changing only the five Heston parameters,

$$v_0,\ \kappa,\ \theta,\ \sigma_v,\ \rho,$$

until the total difference between the model IVs and market IVs is as small as possible. The real data inputs K_i, T_i, and $IV_{\text{market}}(K_i, T_i)$ stay fixed throughout calibration.

6.1 Data

We use market data for SPY, pulled from yfinance. We have one snapshot. The data file is `data/SPY_options_raw.csv`

6.2 Differences Between FFT, GP, and Sparse GP Calibration

All three calibration notebooks solve the same optimization problem. In each case, the optimizer searches over the five Heston parameters

$$(v_0, \kappa, \theta, \sigma_v, \rho)$$

and minimizes the squared difference between model-implied volatilities and market-implied volatilities:

$$\min_{v_0,\kappa,\theta,\sigma_v,\rho} \sum_{i=1}^{N} \left[IV_{\text{model}}(\mathbf{x}_i) - IV_{\text{market}}(K_i, T_i)\right]^2 .$$

The difference between the notebooks is not the calibration objective, but how each notebook computes IV_{model}.

In the FFT calibration notebook, IV_{model} is computed using the Carr–Madan FFT Heston pricer. For each candidate parameter set, the notebook prices options directly under the Heston model. These model option prices are then converted into implied volatilities and compared to the market implied volatilities. This is the most direct calibration method because it evaluates the Heston pricing model itself during each optimizer step. However, it is also the most computationally expensive, since the FFT pricer must be called repeatedly throughout the minimization.

In the GP calibration notebook, the direct FFT pricing step is replaced by a trained Gaussian Process emulator. The emulator takes the input

$$\mathbf{x}_i = \begin{pmatrix} v_0 \\ \kappa \\ \theta \\ \sigma_v \\ \rho \\ K_i \\ T_i \end{pmatrix}$$

and directly predicts the implied volatility:

$$IV_{\text{model}}(\mathbf{x}_i) = IV_{\text{GP}}(\mathbf{x}_i).$$

Thus, the GP notebook does not compute option prices during calibration. Instead, it uses the emulator as a surrogate for the Heston implied-volatility surface. This makes calibration faster than the direct FFT approach, but the result depends on how accurately the GP emulator learned the synthetic Heston training data.

In the sparse GP calibration notebook, the calibration objective is again unchanged, but the emulator is a sparse Gaussian Process rather than a full Gaussian Process. The sparse GP uses a smaller set of representative inducing points instead of the full synthetic training set. Therefore,

$$IV_{\text{model}}(\mathbf{x}_i) = IV_{\text{sparse GP}}(\mathbf{x}_i).$$

This further reduces the computational cost of evaluating the emulator during calibration. The tradeoff is that the sparse GP is an additional approximation: it approximates the full

GP, which itself approximates the Heston implied-volatility surface.
Therefore, the three notebooks differ only in the forward model used inside the optimizer:

$$IV_{\text{model}} = \begin{cases} IV_{\text{FFT}}, & \text{direct Heston FFT calibration,} \\ IV_{\text{GP}}, & \text{full Gaussian Process emulator calibration,} \\ IV_{\text{sparse GP}}, & \text{sparse Gaussian Process emulator calibration.} \end{cases}$$

Below, we show a comparison of Heston parameters obtained from different calibration methods. Although there is difference between the parameter values calibrated with different methods, they yield consistent IV which is shown in the Figure 12.

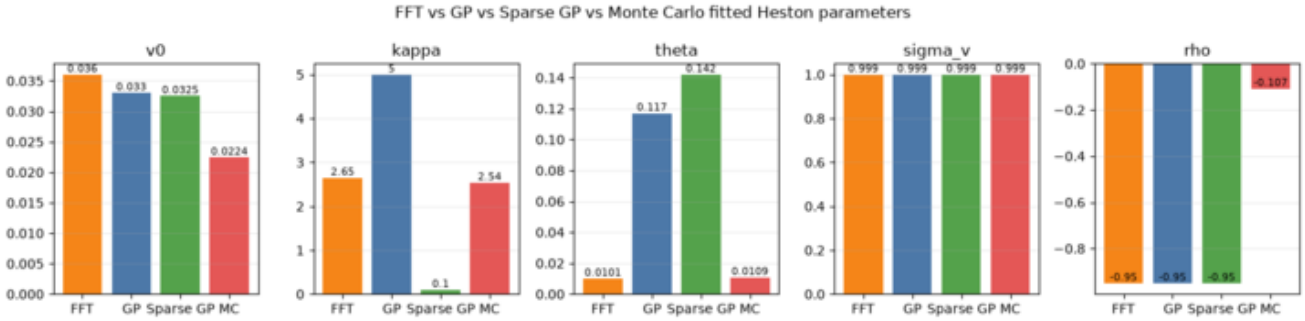

Figure 12: Heston Parameter values obtained after calibration to SPY data. Different color assignments indicate different methods used in calibration.

Similarly, we show a plot of calibration error vs calibration time using different methods in Figure 13 below.

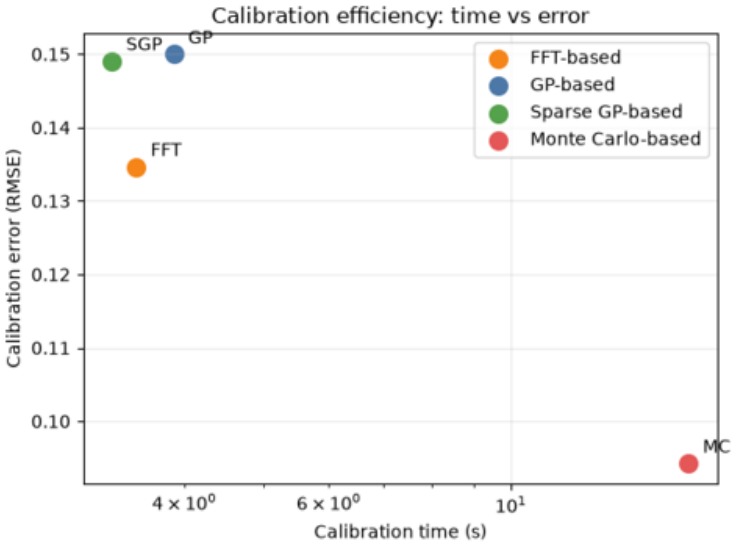

Figure 13: Heston Parameter calibration error vs calibration time. It can be seen that although FFT, GP and SGP calibrate relatively quicky, they yield a larger error in comparison to MC methods.

7 Results

Stochastic processes are inherently non-deterministic. However, by learning patterns from previously generated data, a Gaussian Process (GP) emulator can approximate the underlying relationship and make predictions for unseen inputs. In the Heston model, stochasticity arises from two sources: (1) the stock price evolves according to a stochastic Brownian motion, and (2) the volatility is itself a stochastic process that evolves over time. We trained a Gaussian Process emulator to predict the implied volatility (IV) of an underlying asset as a function of strike price and time to expiration. To accomplish this, we first generated a synthetic dataset by sampling the seven Heston model parameters and computing the corresponding implied volatilities. We then trained both a full Gaussian Process and a sparse Gaussian Process on this dataset to learn the mapping between the model parameters and the implied volatility surface. Finally, we calibrated the emulator using market data for the SPY stock. After calibration, we evaluated the emulator by predicting implied volatilities across a range of strike prices for two different expiration dates and compared the predictions with the observed market implied volatilities. The comparison is shown in Figure 14.

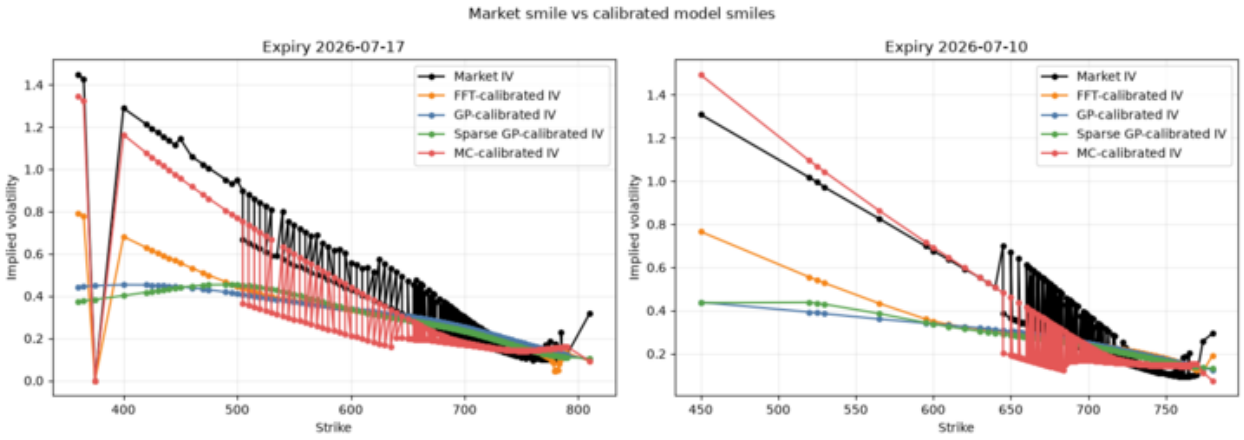

Figure 14: Real market IV compared with the IVs obtained with different calibration methods to the SPY stock at two different maturity dates. We see that MC calibration yields a better approximation and can reproduce the sharp changes in IV. On the other hand, GP, sparse-GP, and FFT calibrated IVs tend to follow a linear behavior in mapping.

References

[1] Peter Carr and Dilip Madan. "Option valuation using the fast Fourier transform". In: *Journal of computational finance* 2.4 (1999), pp. 61–73.

[2] John C Cox, Jonathan E Ingersoll, Stephen A Ross, et al. "A theory of the term structure of interest rates". In: *Econometrica* 53.2 (1985), pp. 385–407.

[3] Steven L Heston. "A closed-form solution for options with stochastic volatility with applications to bond and currency options". In: *The review of financial studies* 6.2 (1993), pp. 327–343.

[4] Wilson Tsakane Mongwe, Rendani Mbuvha, and Tshilidzi Marwala. *Bayesian Machine Learning in Quantitative Finance: Theory and Practical Applications*. Springer, 2025.